

Train-Test Split

Theory & Code for Feature Engineering

Before any feature engineering, scaling, or encoding happens, you must split your data. This is not just a technicality — it is the foundational rule that separates a valid ML pipeline from one that cheats. Get this wrong and every metric you report is a lie.

1. What is Train-Test Split?

You divide your full dataset into two (or three) non-overlapping subsets before touching it:

Split	Purpose	Typical Size
Training Set	Model learns patterns from this data	70–80%
Validation Set	Tune hyperparameters, compare models	10–15% (optional)
Test Set	Final, unbiased evaluation — touch it once only	10–20%

The test set must remain completely untouched until your model is finalized. Looking at it even once — to inform a decision — contaminates it.

2. Why Split BEFORE Feature Engineering?

This is the most commonly violated rule in ML. The correct order is:

Step	Action
1	Split data into Train / Test
2	Fit all transformers (scalers, encoders, imputers) on Train only
3	Transform Train using fitted transformers
4	Transform Test using the SAME fitted transformers (no re-fitting)
5	Train model on transformed Train set
6	Evaluate model on transformed Test set

✓ Correct: Fit StandardScaler on train, transform both train and test with it.

■ Wrong: Fit StandardScaler on the full dataset, then split. The scaler has seen test data — this is data leakage.

Data Leakage means your model has indirectly seen information from the test set during training. It inflates performance on paper and collapses in production.

3. Basic Train-Test Split — Code

3.1 Simple Split (80/20)

```
from sklearn.model_selection import train_test_split import pandas as pd # X =  
features, y = target X_train, X_test, y_train, y_test = train_test_split( X, y,  
test_size=0.2, # 20% for test random_state=42, # reproducibility shuffle=True #  
shuffle before splitting (default) )
```

3.2 Stratified Split (classification problems)

```
# stratify=y ensures class distribution is preserved in both splits # Critical  
when classes are imbalanced (e.g. 95% No, 5% Yes) X_train, X_test, y_train, y_test  
= train_test_split( X, y, test_size=0.2, random_state=42, stratify=y # preserves  
class proportions )
```

3.3 Train / Validation / Test Split

```
# Split into 70% train, 15% validation, 15% test X_train, X_temp, y_train, y_temp  
= train_test_split( X, y, test_size=0.3, random_state=42 ) X_val, X_test, y_val,  
y_test = train_test_split( X_temp, y_temp, test_size=0.5, random_state=42 )  
print(X_train.shape, X_val.shape, X_test.shape)
```

3.4 Verify split sizes

```
print(f'Train: {X_train.shape[0]} rows ({X_train.shape[0]/len(X)*100:.1f}%)')  
print(f'Test: {X_test.shape[0]} rows ({X_test.shape[0]/len(X)*100:.1f}%)') # For  
classification – verify class balance preserved print('Train target  
distribution:') print(y_train.value_counts(normalize=True)) print('Test target  
distribution:') print(y_test.value_counts(normalize=True))
```

4. Correct Feature Engineering Pipeline

Every transformation must be fitted on training data only. Then applied identically to test data using the already-fitted transformer.

4.1 Scaling — StandardScaler

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler() # Fit ONLY on training data scaler.fit(X_train) # Transform both using the SAME fitted scaler X_train_scaled = scaler.transform(X_train) X_test_scaled = scaler.transform(X_test) # Shortcut for train only: X_train_scaled = scaler.fit_transform(X_train) # fit + transform X_test_scaled = scaler.transform(X_test) # transform only
```

4.2 Encoding — OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder enc = OneHotEncoder(handle_unknown='ignore', sparse_output=False) # Fit on train only enc.fit(X_train[['city', 'gender']]) # Transform both train_enc = enc.transform(X_train[['city', 'gender']]) test_enc = enc.transform(X_test[['city', 'gender']]) # handle_unknown='ignore' handles new categories in test gracefully
```

4.3 Imputation — Missing Values

```
from sklearn.impute import SimpleImputer imputer = SimpleImputer(strategy='median') # or 'mean', 'most_frequent' # Fit on train – learns median from train only imputer.fit(X_train) # Transform both – test uses train's median, not its own X_train_imp = imputer.transform(X_train) X_test_imp = imputer.transform(X_test)
```

4.4 Target Encoding (leakage-safe)

```
import category_encoders as ce enc = ce.TargetEncoder(cols=['city']) # Fit on train only (uses y_train to compute means) enc.fit(X_train, y_train) # Transform both X_train['city'] = enc.transform(X_train)['city'] X_test['city'] = enc.transform(X_test)['city']
```

5. The Right Way — sklearn Pipeline

A Pipeline chains all transformers + the model into one object. When you call `pipeline.fit(X_train)`, it fits every step on training data only. When you call `pipeline.predict(X_test)`, it transforms test data using already-fitted steps. Leakage is structurally impossible.

5.1 Numeric + Categorical Pipeline

```
from sklearn.pipeline import Pipeline from sklearn.compose import
ColumnTransformer from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer from sklearn.ensemble import
RandomForestClassifier numeric_cols = ['age', 'fare', 'income'] categorical_cols =
['city', 'gender', 'department'] # Numeric pipeline num_pipeline = Pipeline([
('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler()) ]) #
Categorical pipeline cat_pipeline = Pipeline([ ('imputer',
SimpleImputer(strategy='most_frequent')), ('encoder',
OneHotEncoder(handle_unknown='ignore')) ]) # Combine preprocessor =
ColumnTransformer([ ('num', num_pipeline, numeric_cols), ('cat', cat_pipeline,
categorical_cols) ]) # Full pipeline with model full_pipeline = Pipeline([
('preprocessor', preprocessor), ('model', RandomForestClassifier(n_estimators=100,
random_state=42)) ]) # Fit on train – all steps fitted on train only
full_pipeline.fit(X_train, y_train) # Evaluate on test score =
full_pipeline.score(X_test, y_test) print(f'Test Accuracy: {score:.4f}')
```

✓ The Pipeline guarantees fit happens on train only. No manual fit/transform juggling. No leakage possible.

6. Cross-Validation — Better than a Single Split

A single 80/20 split depends on which rows ended up in which set — this is luck. Cross-validation repeats the split k times, each time using a different fold as the test set. The final score is the average. More reliable, less variance.

Fold	Train Data	Validation Data
1	Folds 2, 3, 4, 5	Fold 1
2	Folds 1, 3, 4, 5	Fold 2
3	Folds 1, 2, 4, 5	Fold 3
4	Folds 1, 2, 3, 5	Fold 4
5	Folds 1, 2, 3, 4	Fold 5
	Final score = mean of all 5 fold scores	

6.1 Basic Cross-Validation

```
from sklearn.model_selection import cross_val_score, KFold kf = KFold(n_splits=5,
shuffle=True, random_state=42) scores = cross_val_score( full_pipeline, # your
pipeline X_train, y_train, # only training data – never touch test cv=kf,
scoring='accuracy' ) print(f'CV Scores: {scores}') print(f'Mean Score:
{scores.mean():.4f}') print(f'Std Dev: {scores.std():.4f}') # low std = stable
model
```

6.2 Stratified K-Fold (for classification)

```
from sklearn.model_selection import StratifiedKFold # Preserves class distribution
in every fold skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score( full_pipeline, X_train, y_train, cv=skf,
scoring='f1_weighted' )
```

7. Common Mistakes

Fitting transformers before splitting

You call `scaler.fit(X)` on the full dataset, then split. The scaler has seen test data. Your test score is optimistic garbage.

Splitting after feature creation from the full dataset

You create a 'mean_age_by_city' feature using all rows, then split. Test rows contributed to their own feature values. Leakage.

Evaluating on training data

Reporting train accuracy as model performance. A model that memorizes training data scores 100% on train and fails completely on new data. Always report test score.

Re-fitting the scaler on test data

Calling `scaler.fit_transform(X_test)` instead of `scaler.transform(X_test)`. Now train and test are scaled differently — the model sees inconsistent inputs.

Ignoring random_state

Not setting `random_state=42` (or any fixed value) means every run gives a different split, making results non-reproducible. Always fix it.

Tiny test sets

Using 5% for test when you have only 500 rows — 25 samples is not enough to reliably estimate performance. Use at least 20% or cross-validation.

8. train_test_split — Parameter Reference

Parameter	Values	What it does
test_size	0.2 (float) or 200 (int)	Size of test set. Float = fraction, int = row count
random_state	Any integer (e.g. 42)	Seeds the random shuffle for reproducibility
shuffle	True / False	Shuffle rows before splitting. Default True
stratify	y or None	Preserves class distribution. Pass y for classification

9. Complete End-to-End Example

```

import pandas as pd from sklearn.model_selection import train_test_split,
cross_val_score from sklearn.pipeline import Pipeline from sklearn.compose import
ColumnTransformer from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer from sklearn.ensemble import
GradientBoostingClassifier from sklearn.metrics import classification_report # 1.
Load data df = pd.read_csv('data.csv') X = df.drop('target', axis=1) y =
df['target'] # 2. Split FIRST – before anything else X_train, X_test, y_train,
y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y ) # 3.
Define feature groups num_cols =
X.select_dtypes(include='number').columns.tolist() cat_cols =
X.select_dtypes(include='object').columns.tolist() # 4. Build pipeline num_pipe =
Pipeline([('imp', SimpleImputer(strategy='median')), ('scl', StandardScaler())])
cat_pipe = Pipeline([('imp', SimpleImputer(strategy='most_frequent')), ('enc',
OneHotEncoder(handle_unknown='ignore'))]) pre = ColumnTransformer([('num',
num_pipe, num_cols), ('cat', cat_pipe, cat_cols)]) pipeline = Pipeline([('pre',
pre), ('model', GradientBoostingClassifier(random_state=42))]) # 5. Cross-validate
on train only cv_scores = cross_val_score(pipeline, X_train, y_train, cv=5,
scoring='f1_weighted') print(f'CV F1: {cv_scores.mean():.4f} +/-
{cv_scores.std():.4f}') # 6. Final fit on full train set pipeline.fit(X_train,
y_train) # 7. Evaluate on test – ONE TIME ONLY y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred))

```

10. Summary

Rule	Why it matters
Split before any transformation	Prevents data leakage
Fit transformers on train only	Prevents leakage into test
Transform test with train-fitted objects	Consistent preprocessing
Use Pipeline to enforce order	Leakage structurally impossible
Use stratify=y for classification	Balanced class distribution
Fix random_state	Reproducible results
Never touch test set during development	Unbiased final evaluation
Cross-validate on train only	Reliable performance estimate

The golden rule: fit on train, transform both. Everything else follows from this.